REFACTORIZACIÓN DE CÓDIGO

Entornos de Desarrollo

Marco Valiente
DAW1

Contenido

Introducción 2

Desarrollo 3

1. Eliminación de Código no deseado 3
2. Refactorización de Código repetido 4

Introducción

Para esta práctica, vamos a ver un par de funciones dentro de antiguos proyectos de Java donde podamos hacer refactorización de código, tomar el código repetido y limpiarlo para hacerlo más utilizable y que ocupe menos espacio, mejorando la legibilidad del mismo.

Desarrollo

1. Eliminación de Código no deseado

En este programa en Java, tenemos el siguiente código sin usar que se queda guardado después de un inicial intento de querer tener un sistema de confirmación si desea guardar o no.

```
//Holders de Objetos
VideoDaw miVideoClub;
boolean allSaved = true;

//Iniciali

//Creacion
if (colecc
    regist
    allSaved = false;
}

//Seleccion por defecto
miVideoClub = coleccionVideoClubs.get(0);
```

Remove local variable 'allSaved'
Inspection 'Reassigned variable' options
Split into declaration and assignment
Convert to atomic
Press to toggle preview

```
112 boolean allSaved = true;
119 allSaved = false;
157 allSaved = false;
240 allSaved = false;
287 allSaved = false;
330 allSaved = false;
377 allSaved = false;
435 allSaved = false;
487 allSaved = false;
```

Podemos hacer click en “Remove Local Variable” para quitar esta variable de todos los lugares de donde está definida.

```
//Holders de Objetos
VideoDaw miVideoClub;

//Inicializacion de Archivos

//Creacion VideoClub inicial
if (coleccionVideoClubs.isEmpty()) {
    registroDeVideoDaw(coleccionVideoClubs);
}
```

¡Y listo! Como se puede ver, usar esta función de IntelliJ permite quitar de todos los lugares de donde se esté usando esta variable de forma casi instantánea. Este tipo de medidas nos ahorran tiempo.

2. Refactorización de Código repetido

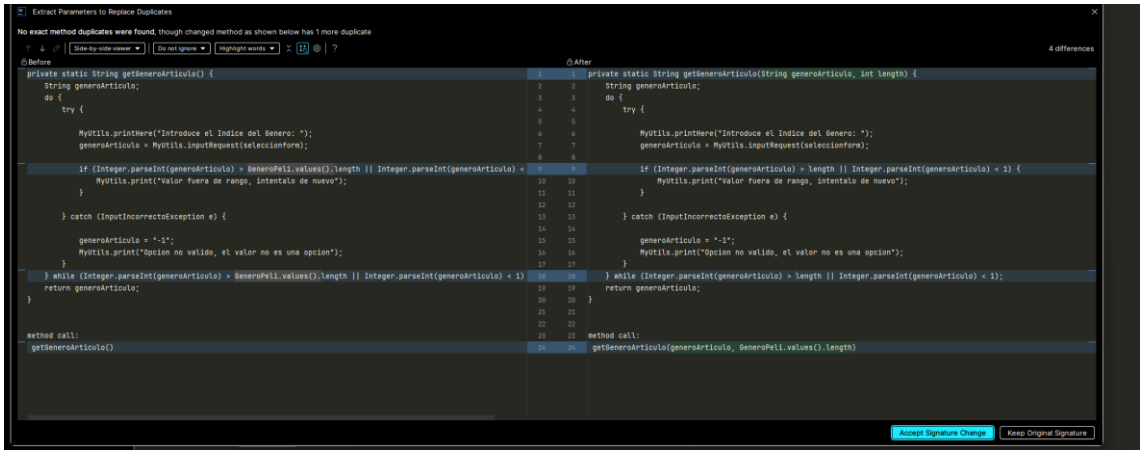
Con esto visto, ahora vamos a buscar una pieza de código que se repita por varias partes, y que podamos factorizar o hacer factor común.

```
//Case 1 - Pelicula
case "1":
    do {
        try {

            MyUtils.printHere(text: "Introduce el Indice del Genero: ");
            generoArticulo = MyUtils.inputRequest(seleccionform);

            if (Integer.parseInt(generoArticulo) > GeneroPeli.values().length || Integer.parseInt(generoArticulo) < 1) {
                MyUtils.print("Valor fuera de rango, intentalo de nuevo");
            }

        } catch (InputIncorrectoException e) {

            generoArticulo = "-1";
            MyUtils.print("Opcion no valido, el valor no es una opcion");
        }
    } while (Integer.parseInt(generoArticulo) > GeneroPeli.values().length || Integer.parseInt(generoArticulo) < 1);

    GeneroPeli generoPeli = GeneroPeli.values()[Integer.parseInt(generoArticulo) - 1];

    //Creacion Objeto
    nuevoArticulo = new Pelicula(titulo, generoPeli);

    break;


//Case 2 - VideoJuego
case "2":
    do {
        try {

            MyUtils.printHere(text: "Introduce el Indice del Genero: ");
            generoArticulo = MyUtils.inputRequest(seleccionform);

            if (Integer.parseInt(generoArticulo) > GeneroVideoJuego.values().length || Integer.parseInt(generoArticulo) < 1) {
                MyUtils.print("Valor fuera de rango, intentalo de nuevo");
            }

        } catch (InputIncorrectoException e) {

            generoArticulo = "-1";
            MyUtils.print("Opcion no valido, el valor no es una opcion");
        }
    } while (Integer.parseInt(generoArticulo) > GeneroVideoJuego.values().length || Integer.parseInt(generoArticulo) < 1);

    GeneroVideoJuego generoVideoJuego  = GeneroVideoJuego.values()[Integer.parseInt(generoArticulo) - 1];

    //Creacion Objeto
    nuevoArticulo = new VideoJuego(titulo, generoVideoJuego);

    break;
```

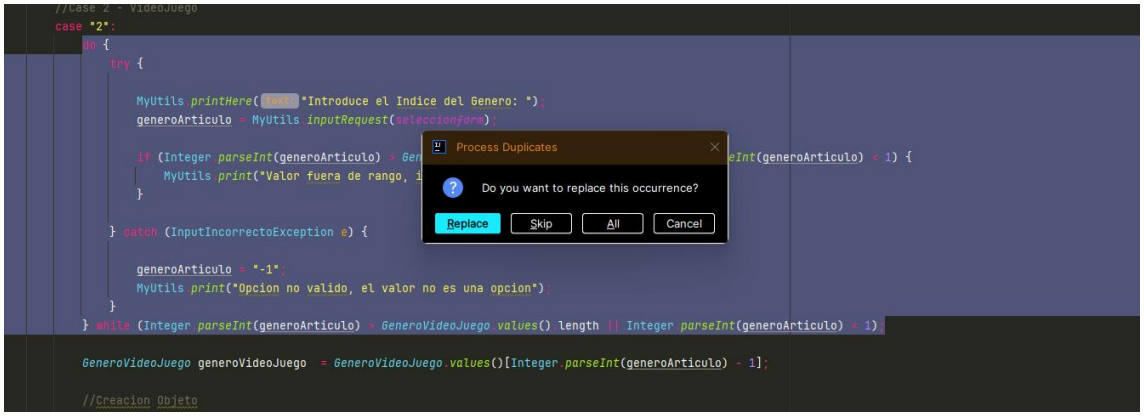
Tenemos este código, que es similar entre sí, el único detalle que cambia es que se usa en el Enum GeneroPeli en uno, y GeneroVideoJuego en el otro.

Para esto, podemos usar una función genérica para Java a la hora de insertar variables, pero una solución más sencilla, es pasarle los valores de rango que un loop deberá de cumplir.

Para realizar esto, vamos a ir a refactor y a extraer método.



Y nos pregunta realizar el cambio. IntelliJ está detectando que es un factor común la longitud del artículo, por lo que le damos a realizar cambios.



Y después de esto, nos pregunta si queremos reemplazar esta ocurrencia aquí, a lo que le decimos que sí.

```java
}

private static String getGeneroArticulo(String generoArticulo, int length) {  2 usages  new *
    String generoArticulo;
    do {
        try {

            MyUtils.printHere( text: "Introduce el Indice del Genero: ");
            generoArticulo = MyUtils.inputRequest(seleccionform);

            if (Integer.parseInt(generoArticulo) > length || Integer.parseInt(generoArticulo) < 1) {
                MyUtils.print("Valor fuera de rango, intentalo de nuevo");
            }

        } catch (InputIncorrectoException e) {

            generoArticulo = "-1";
            MyUtils.print("Opcion no valido, el valor no es una opcion");
        }
    } while (Integer.parseInt(generoArticulo) > length || Integer.parseInt(generoArticulo) < 1);
    return generoArticulo;
}

public static void registroDeVideoDaw(List<VideoDaw> coleccion) {  2 usages  COMPUS021\daw1
```

No nos interesa, sin embargo, que géneroArtículo sea un parámetro, por lo que borramos a este de ser un input del método.

```java
    //Seleccion del Genero basado en tipo
    switch (generoArticulo) {

        //Case 1 - Pelicula
        case "1":
            generoArticulo = getGeneroArticulo(GeneroPeli.values().length);

            GeneroPeli generoPeli = GeneroPeli.values()[Integer.parseInt(generoArticulo) - 1];

            //Creacion Objeto
            nuevoArticulo = new Pelicula(titulo, generoPeli);

            break;


        //Case 2 - VideoJuego
        case "2":
            generoArticulo = getGeneroArticulo(GeneroVideoJuego.values().length);

            GeneroVideoJuego generoVideoJuego  = GeneroVideoJuego.values()[Integer.parseInt(generoArticulo) - 1];

            //Creacion Objeto
            nuevoArticulo = new VideoJuego(titulo, generoVideoJuego);

            break;

    }

    return nuevoArticulo;
}

private static String getGeneroArticulo(int length) {  2 usages  new *
    String generoArticulo;
    do {
        try {

            MyUtils.printHere( text: "Introduce el Indice del Genero: ");
            generoArticulo = MyUtils.inputRequest(seleccionform);

            if (Integer.parseInt(generoArticulo) > length || Integer.parseInt(generoArticulo) < 1) {
                MyUtils.print("Valor fuera de rango, intentalo de nuevo");
            }

        } catch (InputIncorrectoException e) {

            generoArticulo = "-1";
            MyUtils.print("Opcion no valido, el valor no es una opcion");
        }
    } while (Integer.parseInt(generoArticulo) > length || Integer.parseInt(generoArticulo) < 1);
    return generoArticulo;
}
```

Borramos de ambos lados el dato inicial, y nuestra refactorización ha sido completada. Podemos ahora darle a “rename” para poner un nombre que no confunda esta función con los Getter de una clase.

```
        return nuevoArticulo;
    }

    private static String getGeneroArti
        String generoArticulo;
        do {
            try {

                MyUtils.printHere( text
                generoArticulo = MyUtil

                if (Integer.parseInt(ge
                    MyUtils.print("Valo
                }
```

```
Show Context Actions          Alt+Enter
Paste                         Ctrl+Shift+V
Copy / Paste Special          >
Column Selection Mode         Alt+Shift+Insert
Find Usages                   Alt+F7
Go To                         >
Folding                       >
Analyze                       >
Rename...                     Shift+F6
Refactor                      >
Generate...                   Alt+Insert
```

Para esto la encontramos y usamos rename.

```
    switch (generoArticulo) {

        //Case 1 - Pelicula
        case "1":
            generoArticulo = pedirGenero(GeneroPeli.values().length);

            GeneroPeli generoPeli = GeneroPeli.values()[Integer.parseInt(generoArticulo) - 1];

            //Creacion Objeto
            nuevoArticulo = new Pelicula(titulo, generoPeli);

            break;


        //Case 2 - VideoJuego
        case "2":
            generoArticulo = pedirGenero(GeneroVideoJuego.values().length);

            GeneroVideoJuego generoVideoJuego  = GeneroVideoJuego.values()[Integer.parseInt(generoArticulo) - 1];

            //Creacion Objeto
            nuevoArticulo = new VideoJuego(titulo, generoVideoJuego);

            break;

    }

    return nuevoArticulo;
}

private static String pedirGenero(int length) {  2 usages  new *
    String generoArticulo;
    do {
        try {

            MyUtils.printHere( text: "Introduce el Indice del Genero: ");
            generoArticulo = MyUtils.inputRequest(seleccionform);
```

Escribimos el nombre, y es cambiada en todos los lugares donde estuvo escrito.